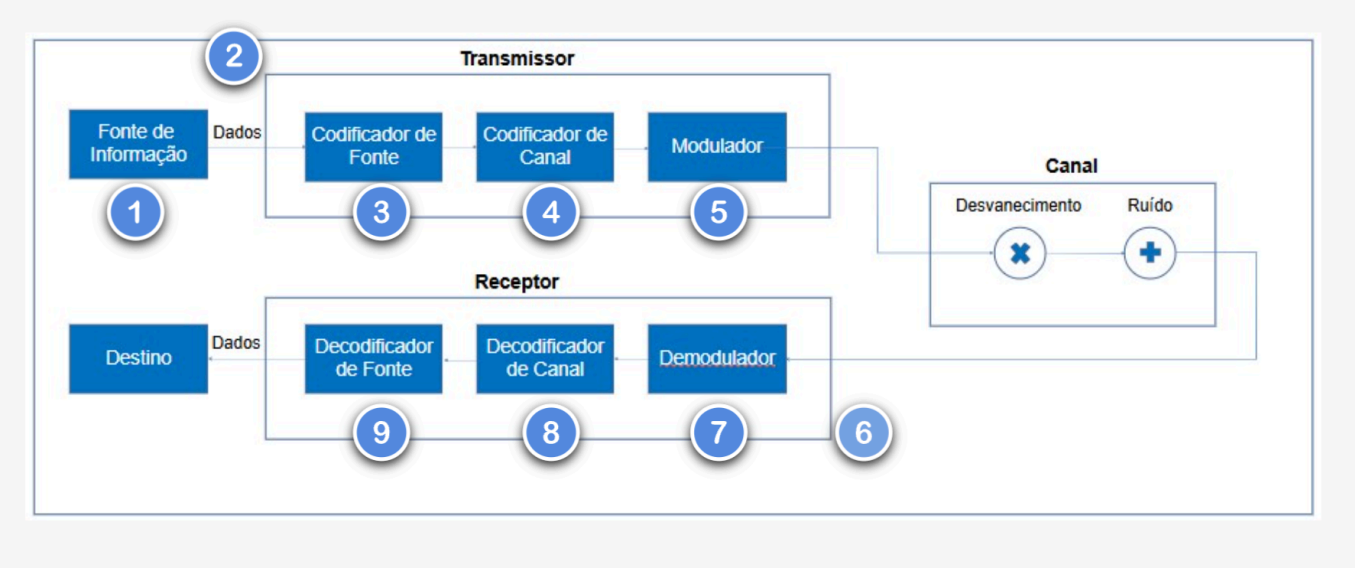
- Reduzir drasticamente a quantidade de bits necessários para transmitir ou armazenar informação sem perder (ou perdendo controladamente) os dados
- Reduzir bits mantendo (ou quase) a informação original (comprimir)
- O objetivo de todo sistema de comunicação é transmitir mensagens de um ponto A para B, e existem várias técnicas de adaptação, sendo algumas delas: Modulação e codificação da fonte (compressão)
- O transmissor é composto por um codificador de fonte, um codificador de canal e um modulador

Diagrama em blocos de um sistema digital



- Onde:

```
Fonte → Codificação de Fonte → Codificação de Canal → Modulador → Canal →
Demodulador → Decodificação de Canal → Decodificação de Fonte → Destino
```

```
- Fonte: De onde sai a informação original, a mensagem que será enviada
- Codificação de Fonte (compressão): Reduz o número de bits necessários para
representar a mensagem, eliminando redundância. Pode ser sem perda (Huffman,
LZ, RLE) ou com perda (JPEG, MP3, vídeo). Papel: economiza banda e
armazenamento. É como fazer uma mala, dobramos as roupas para caber mais
coisas em menos espaço
- Codificação de Canal: Adiciona bits de redundância (paridade, códigos
corretores) para proteger contra erros causados pelo canal. Papel: torna a
transmissão mais robusta contra ruído, interferência e desvanecimento.
- Modulador: Converte os bits (0s e 1s) em uma forma de onda que pode viajar
pelo canal (amplitude, frequência ou fase da portadora). Papel: adapta o
```

```
sinal ao meio físico (ar, cabo, fibra)
- Canal: O meio de transmissão (ar, cabo de cobre, fibra óptica, satélite).
Sofre ruído, interferência, atenuação, desvanecimento etc. Analogia: é a
estrada (pode ter buracos, chuva, engarrafamento)
- Demodulador: Recebe a onda do canal e tenta recuperar os bits originais
(inverte a modulação). Papel: extrai os bits do sinal recebido.
- Decodificação de Canal: Usa os bits de redundância para corrigir ou
detectar erros que ocorreram no canal. Papel: limpa os erros antes de
entregar a mensagem
- Decodificação de Fonte Reverte a compressão feita na codificação de fonte,
recuperando a mensagem original (no caso sem perda) ou a versão aproximada
(com perda). Papel: devolve a informação no formato que o destino entende
- Destino: - O receptor final (celular, TV, computador, pessoa, etc.) que
consome a informação.
```

Fonte discreta sem memória (FDSM)

- É o modelo mais simples de fonte
- Símbolos de um alfabeto finito (pixels, letras, bits)
- A probabilidade de um símbolo não depende dos anteriores (Não possui memória)
- Variável aleatória discreta S = {s0, s1, s2....sk-1} onde S é o conjunto de simbolos possiveis, com probabilidade pk:
  - $\sum_{k=0}^{K-1} p_k = 1$
- Quantidade de informação de um símbolo (o famoso conceito de Shannon):

$$I(s_k) = \log_2\left(\frac{1}{p_k}\right) \quad [\text{bits}]$$

- Quanto mais raro o símbolo (menor pk), maior a surpresa e mais informação ele carrega
- Propriedades:
  1. $I(s_k) = 0$ se pk=1 (certeza, zero informação) (Se o símbolo é certo(pk = 1))
  2. $I(sk) \geq 0$ (Informação nunca é negativa)
  3. Se $pk < pl$ então $I(sk) > I(sl)$ (Quanto menor a probabilidade, maior a informação (mais raro = mais informativo))
  4. Informação conjunta: $I(sksl) = I(sk) + I(sl)$ (A informação de dois símbolos independentes é a soma das informações individuais)
     - I(s_k) é a quantidade de informação (ou auto-informação) do símbolo sk
     - Quanto **mais comum** (maior pk), menor I(s_k)

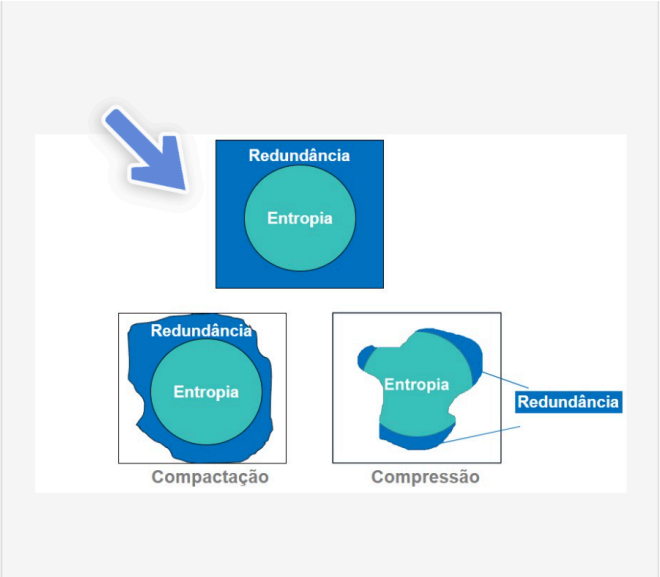
Entropia H(S)

- A entropia é uma medida de incertezas ou surpresa de uma fonte de informação

- Calculo da entropia:
  $H(S) = \sum_{k=0}^{K-1} p_k \log_2 \left(\frac{1}{p_k}\right) \quad [\text{bits/símbolo}]$
- É o limite teórico de compressão sem perda (é o menor tamanho possível (em bits) que um código sem perda pode alcançar.)
- $0 \leq H(S) \leq log2K$'
  - Mínimo possível = 0
  - Máximo possível = $log2K$ (onde K é o número total de símbolos diferentes)
- H(S)=0 só se um símbolo tem probabilidade 1
- $H(S) = log2K$ só se todos os símbolos são equiprováveis (todos têm a mesma probabilidade $\frac{1}{K}$)

Diferença entre compactação e compressão

- Compactação = redução geral de tamanho (não altera a entropia (não perde informações) (reduz a redundância))
- Compressão = permite descartar informação imperceptível (ex: JPEG, MP3) (altera redundância e entropia)



- O que é redundância nesse caso:
  - É tudo que está acima do limite de bits (Entropia)
  - Redundância = informação extra, previsível e repetitiva que a mensagem carrega. A codificação de fonte serve exatamente para tirar essa redundância e transmitir menos bits, gastando menos banda e tempo

Requisitos para códigos em sistemas digitais

- Dois requisitos obrigatórios:
  - Palavras-código devem ser binárias (0 e 1)

    - Devem ser inequivocamente decodificáveis (prefix-free ou uniquely decodable) (Ou seja, cada sequência de bits deve corresponder a um único símbolo e não pode haver ambiguidade)
- Palavra código:
    - É o “nome” binário que damos para cada símbolo depois da compressão. A codificação de fonte transforma os símbolos da fonte (letras, pixels, amostras de áudio etc.) em sequências binárias curtas e eficientes
    - é a sequência binária que representa um símbolo após a codificação de fonte.
    - Quanto mais provável o símbolo, normalmente mais curta é sua palavra-código (é o que o Huffman faz)
    - O conjunto completo de palavras-código forma o código da fonte.
- Comprimento médio das palavras-código é dado pela equação:
  $\bar{L} = \sum_{k=0}^{K-1} p_k \, l_k$
    - Onde lk = comprimento em bits da palavra-código do símbolo

1º Teorema de Shannon (Teorema de Codificação de Fonte)

- Estabelece o limite minimo
- O limite teórico:
  $\bar{L} \geq H(S)$
    - Qualquer código que consiga $\bar{L}$ muito próximo de H(S) é considerado ótimo.
- Eficiência da codificação:
  $\eta = \frac{H(S)}{\bar{L}}$
- η representa a porcentagem se η=1 significa 100% codificação ótima
- L é o comprimento médio das palavras-código
- E H(S) é entropia

Códigos prefixos

Códigos prefixos

É um tipo de código de compressão onde nenhum código de uma sequência pode ser o prefixo de outro código dentro do mesmo conjunto de códigos. Em outras palavras, ao decodificar uma mensagem, o decodificador pode identificar o fim de cada símbolo de forma imediata e sem ambiguidade, pois nenhum código interfere na leitura dos outros.

Símbolo	Probabilidade	Código I	Código II	Código III
S_0	0,5	0	0	0
S_1	0,25	1	10	01
S_2	0,125	00	110	011
S_3	0,125	11	111	0111

Códigos prefixos

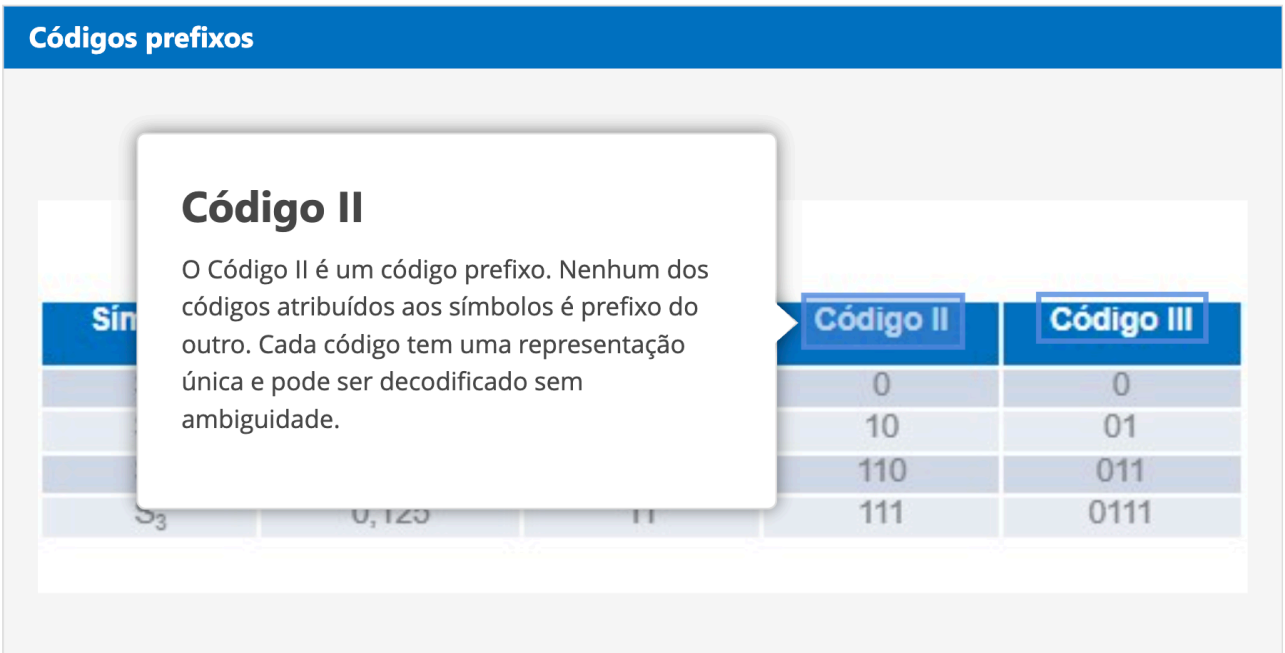
Código I

O Código I não é um código prefixo, porque o código de S0 é o prefixo do código de S2. Como não é possível diferenciar os dois durante a decodificação, o Código I é ambíguo, ou seja, não é um código prefixo.

Código I	Código II	Código III
0	0	0
1	10	01
00	110	011
11	111	0111

- Temos o 0 e o 1 "sozinhos" e depois 00 (começa com 0) e 11 (começa com 1), os sozinhos não podem ser prefixo (primeiro número) dos outros códigos do conjuntos. O código 3

também não é prefixo pois tem 0 sozinho e todos os outros começam com 0



- É prefixo pois nenhum outro começa com 0

Codificação de Huffman

- Técnica sem perda que atribui códigos curtos aos símbolos frequentes
- Passo a passo:
  - Ordene símbolos por probabilidade (menor para maior)
  - Some os dois menores -> crie nó pai
  - Repita até restarem dois nós
  - Atribua 0 e 1 nas ramificações (de baixo para cima)
- Tirar print do exemplo do slide
- Raiz -> ramifica em 0 e 1
- Folhas são os símbolos com seus códigos
- códigos de comprimento variável baseados em probabilidade

Codificação Aritmética

- Mais eficiente que Huffman quando probabilidades são muito desiguais
- Em vez de códigos fixos, representa toda a mensagem como um único número real entre 0 e 1
- Cada símbolo vai subdividindo o intervalo proporcionalmente à sua probabilidade
- fontes com muitos símbolos

Codificação Lempel-Ziv (LZ77 / LZ78)

- Algoritmo adaptativo (não precisa conhecer probabilidades antes)

- Base de ZIP, PNG, GIF, etc

Codificação Run-Length (RLE)

- Simples e excelente para dados com longas sequências repetidas (ex: imagens com áreas sólidas)

Técnicas com perda

- DCT (Transformada Discreta de Cosseno): usada em JPEG e MPEG. Transforma pixels em frequências descarta altas frequências (imperceptíveis ao olho)
- Fator de Quantização (NQuant): controla o quão agressiva é a quantização.
    - Baixo NQuant = alta qualidade, arquivo grande
    - Alto NQuant = arquivo pequeno, qualidade cai

Benefícios da codificação de fonte

- Reduz largura de banda necessária
- Reduz tempo de transmissão
- Economiza armazenamento
- Mantém qualidade quando usada sem perda
- é essencial para comunicação eficiente e econômica